3 SIGIL · PRESENTER CUE SHEET

Why proofs can be forged, and our fix

Sami Daire
about 5 minutes
demos C–F + Verify

BEFORE YOUR TURN

The app is open at `http://127.0.0.1:8765` → **Open dashboard**. In the left sidebar, **C, D, E and F** each have a ✓. If a page says "Nothing has run yet", press its Run button once (C and D ≈ 3 s, E ≈ 10 s). The bottom bar says **Idle**. The ← → keys move between demos. Numbers change a little with each live run, so read them off the screen. If something breaks: **Show saved results** in the sidebar, or the backup `python demo/server.py --simulate --port 8766`.

1 Start the long run first · Demo F

CLICK Sidebar **F · Protocol v2** → **Run demo F live** (top right). Takes about 80 s.

CHECK The console bar at the bottom starts moving.

SAY *"This makes a brand-new proof right now. While it runs, I'll show you why we need it."*

2 The problem · Demo C, Ownership forgery

CLICK Sidebar **C · Ownership forgery**.

SHOW Owner: **0/64** bits wrong. Thief: also **0/64**, and the message reads **MALLORY!** The "never watermarked" table: every row says **False claim works**.

SAY *"Old methods let you pick the key after seeing the model. A thief makes a fake key that finds their own name in our model with zero errors, even in models nobody marked. So reading a mark alone proves nothing."*

3 Our fix: lock the seed first · Demo D

CLICK → or sidebar **D · Committed seed**.

SHOW Naive fix: **1,000/1,000** fakes pass. Real fix: **0/100,000** pass, best thief still **16/64** wrong. On the chart every try is far from the green "accept" line.

SAY *"The owner publishes a hash of a secret seed at registration; the key and the mark both come from it. Now the thief must guess the seed. 100,000 tries, none came close. An even chance needs about 2.5 billion tries, around 196 hours on one CPU core. At 256 bits it is out of reach."*

4 Hiding the secret isn't enough · Demo E

CLICK → or sidebar **E · Zero-knowledge proofs**.

SHOW E1: owner **and** thief both get "Proof verified: true". E2: the owner's and the thief's verification keys are **identical**.

SAY *"A zero-knowledge proof hides the secret. But with the EZKL tool the thief's fake proof passes too, under the owner's own key. A valid proof alone doesn't say who the owner is."*

5 The full system · Demo F (the run is done now)

CLICK → or sidebar **F · Protocol v2**.

CHECK "Ownership verified · 0 of 64 bits revealed" and every check has a green tick.

SHOW Prove ≈ **4 s**, proof ≈ **880 bytes**, owner **0/64** wrong. F3: thief "no proof possible". F4: all four misuses **Rejected**.

SAY *"Our circuit rebuilds the key and mark from the locked seed inside the proof, allows at most 8 wrong bits, and is tied to this exact file and this one claim. The proof is under 1 KB; EZKL's is 60 KB. The thief has no seed: 43 of 64 bits wrong, so no proof."*

6 The final check · Verify ownership

CLICK Sidebar **Verify ownership**.

SHOW The **Ownership proven** stamp and the list of checks, all ticked. Scroll down to the proof itself.

SAY *"That's the whole proof: 3 points, under 1 KB, and not one watermark bit is in it. Anyone can check it with the public key; the watermark never leaves the owner."*

7 Try to cheat, then on-chain · still on Verify ownership

CLICK Each **Try a misuse** button, one by one (about 1 s each).

CHECK The console marks the failing check with `[XX]`.

SAY *"Each button changes one thing. The verifier catches every one, each with its own reason."*

replayed for another claim	made for a different claim
checked against a different model	about a different model
tampered proof bytes	the proof doesn't verify
receipt from an untrusted registry	the receipt isn't trusted

LAST Click the **On-chain** tab and show **243,225 gas**. After a live F run it says the proof was made earlier. That is expected.

SAY *"The same proof is also checked by a smart contract on a local blockchain."*

HAND-OFF TO YASH

"Yash will show what runs behind the screen, who wins when two people claim the same model, and how this would be used for real."

IF ASKED: "WHAT DOES THE VERIFIER LEARN?"

"Only that the registered owner's mark is in this exact file, with at most 8 of 64 bits wrong. Nothing about the seed, the key or the mark."